D03 · 정책서

promptbench

예산 · 승인 · 장부 · 채점 · 기본값 선정 규칙 — 규칙 · 이유 · 예시. 선정 규칙은 실행 전에 사전 등록한다.

문서 버전	1.0
작성일	2026-10-20
작성자	김태민
문서 상태	구현·실측 완료본

변경 이력

버전	일자	내용	작성자
1.0	2026-10-20	최초 작성	김태민

목차

1. 정책 목록
2. 1건당 비용 예시와 단계별 예산 — 계산 결과
3. 사전 등록표 — δ 와 부트스트랩
4. 선정 규칙 가상 예시 — choose_default 실행 결과
5. 예산·장부
6. 승인 게이트
7. 비용 추정
8. 배치 분할·정지
9. 실패 처리
10. 채점
11. 기본값 선정
12. 데이터 라이선스
13. 인증·비밀
14. 가격표 관리

이 문서의 역할 — promptbench 가 사람 대신 내리는 결정(얼마까지 쓸지 · 언제 멈출지 · 실패를 어떻게 셀지 · 어떤 조합을 기본값으로 고를지)을 정책 하나에 규칙 · 이유 · 예시로 적는다. 규칙은 구현(계획 B)의 기준이고, 이유는 나중에 규칙을 바꿀 때 무엇을 잃는지 알려 준다. 정책 ID(`POL-nn`)는 D02 요구사항의 "관련 정책"과 연결되며, 각 정책 표의 "관련 요구사항"은 D02 에서 자동으로 모은 것이다.

표기 원칙 — 정확도 · 비용 · 지연의 실측 값은 D09 에 있고, 이 문서의 금액은 실측 전에 정한 목표치다. 2장 · 4장의 숫자는 손으로 쓰지 않고 빌드할 때 `promptbench.pricing` · `promptbench.selection` · `tools/budget.py` 를 실행해 넣으며, `tests/test_d03.py` 가 같은 입력으로 같은 값을 고정한다. 4장의 30건 결과는 고정 시드로 만든 **가상 예시**다. Anthropic API 동작 중 구현 1단계에서 실제로 확인한 것은 `구현 1단계에서 확인함` 으로 표시한다(POL-02 · POL-05).

이 문서에서 정한 것 — (1) spec §13 의 미결정 "배치 하나의 크기": 요청 200건 또는 보수 금액 $0.50 중 먼저 닿는 쪽(초기값, 실측 후 조정) → POL-20. (2) T3 채점자의 "보정"은 점수를 고치는 일이 아니라, 교정 일치율이 80% 미만일 때 결과에 "참고용" 표시를 붙이는 기준이다 → POL-12. (3) 채점자 쪽 실패는 후보의 오답이 아니라 "채점 불가"로 따로 센다 → POL-09 · POL-22. (4) 선정 규칙의 δ 와 부트스트랩 설정은 계획 해시에 들어가고, run 시작 뒤 δ 가 바뀌면 `report` 가 선정을 거부한다 → POL-13. (5) 고유명사 로마자 표기는 형식 항목에서 허용한다(D08 의 결정) → POL-22. (6) 재시도는 인프라 실패와 채점 불가만 — 거절 · 잘림 · 파싱 실패는 확정 오답 → POL-10. (7) T3 짝지은 비교는 채점된 항목의 교집합에서, 20건 미만이면 판정 보류 · 1,000건당 비용의 정의 → POL-14. (8) stage 별 줄이는 순서 — R1 은 Haiku 비공통 표본, latency 는 3 → 2 → 1건, pilot · R2 · 채점은 줄이지 않음 → POL-21. (9) 채점자는 Sonnet 5.5 · high, 채점은 T3 공통 30건만(2026-10-20 사용자 결정 — 예산 상한 $5) → POL-22 · 2장.

1. 정책 목록

영역	정책
예산·장부	POL-01 실측 전체 예산 상한 5달러 POL-02 실제 비용은 응답 usage 로 재계산해 장부에 기록
승인 게이트	POL-03 승인 전 과금 호출 0건 POL-04 승인은 계획 해시에 묶이고 승인된 계획만 실행
비용 추정	POL-05 예상 비용은 count_tokens 로 계산해 예상·보수·최악 3종으로 낸다 POL-06 예상치와 실제 비용을 비교해 보고 POL-21 통과 기준은 보수 ≤ 남은 예산(파일럿은 최악), 넘으면 정해진 순서로 줄인다
배치 분할·정지	POL-07 배치는 순서대로 제출하고 다음 배치가 상한을 넘기면 제출 전 정지 POL-08 정지 시 남은 요청은 pending 으로 보존 POL-20 배치 하나는 요청 200건 또는 보수 금액 $0.50 중 먼저 닿는 쪽까지
실패 처리	POL-09 실패 항목은 오답 처리하고 유형별 건수 보고 POL-10 자동 재시도 없음 — 재시도는 새 계획을 만들어 다시 승인
채점	POL-11 업무별 품질 지표로 채점 POL-12 교정 일치율이 80% 미만이면 T3 결과를 참고용으로 표시 POL-22 채점자는 고정하고 블라인드로 채점하며, 채점자 쪽 실패는 채점 불가로 센다
기본값 선정	POL-13 선정 규칙과 δ 는 실행 전에 사전 등록하고 결과를 본 뒤 바꾸지 않는다 POL-14 동급 후보 중 1000건당 비용이 가장 낮은 조합 선택
데이터 라이선스	POL-15 데이터 원본은 커밋하지 않고 라이선스와 출처를 기록
인증·비밀	POL-16 ANTHROPIC_API_KEY 만 사용 POL-17 키는 로그·DB·리포트에 남기지 않음
가격표 관리	POL-18 가격표는 설정 파일로 두고 확인일 기록 POL-19 확인일이 30일을 넘으면 승인 화면에 경고

2. 1건당 비용 예시와 단계별 예산 — 계산 결과

2-1. 1건당 비용 — request_cost

입력 1,000토큰 · 출력(thinking 포함) 700토큰인 요청 1건의 비용이다. 가격표는 `config/pricing.yaml` (확인일 2026-10-20)이며, 아래 숫자는 빌드할 때 `request_cost` 를 실행해 얻은 값이다. 토큰 수는 spec §4-1 의 설계 단계 추정이라 실제와 다를 수 있다 — 파일럿 뒤 계획기가 실제 값으로 다시 계산한다(POL-05).

모델	동기 호출 1건	Batch 1건	Batch 1,000건
claude-haiku-5-5	$0.000450	$0.000225	$0.2250
claude-sonnet-5-5	$0.009000	$0.004500	$4.5000
claude-opus-5-5	$0.018000	$0.009000	$9.0000

Batch 는 동기 호출의 절반이다(할인율 50%). 모델 사이의 가격 차이가 크다 — 같은 요청이 Haiku 와 Opus 사이에서 40배 벌어지므로, 품질이 같다면 싼 쪽을 고르는 선정 규칙(POL-14)의 의미가 생긴다.

2-2. 단계별 예산 — 배분 × 가격표

POL-01 의 단계별 금액이다. spec 의 배분(요청 수)에 2-1 의 단가를 곱해 빌드할 때 계산한다 — 손으로 쓴 목표가 아니다.

순서	단계 (plan --stage)	배분	요청 수	계산 금액
1	pilot	실제로 쓰는 (모델 × effort) 8개 × 3 업무 × 3건 · v1	72 (Haiku 45 · Sonnet 18 · Opus 9)	$0.17 목표치
2	R1	Haiku 5단계 × main 100건 · v1 · v2	3,000 (Haiku 3,000)	$0.67 목표치
3	judge-R1	R1 의 T3 공통 30건 요약 채점(Sonnet 5.5 · high) — R2 의 이긴 버전 판정에 필요	300 (Sonnet 300)	$1.29 목표치
4	R2	이긴 버전 하나로 Sonnet low/medium · Opus low × 공통 30건	270 (Sonnet 180 · Opus 90)	$1.62 목표치
5	judge-R2	R2 의 T3 공통 30건 요약 채점(Sonnet 5.5 · high)	90 (Sonnet 90)	$0.39 목표치
6	latency	조합(업무 × 모델 × effort × 버전) 39개 × 공통 앞 3건 · 동기 · 할인 없음	117 (Haiku 90 · Sonnet 18 · Opus 9)	$0.36 목표치
—	**합계**		3,849	**$4.51** 목표치
—	예비	실패 재시도 · 채점 불가 재채점 · 추정 오차 — 단계에 배분하지 않음		$0.49 목표치
—	상한 (POL-01)			$5.00 — 합계가 안에 든다

가정 — 후보 요청 입력 1,000 · 출력(thinking 포함) 700토큰(spec §4-1), 채점 요청 입력 1,800 · 출력 500토큰(Sonnet 5.5 · high — 기사 · 참조 요약 · 후보 요약 · 루브릭을 넣는다고 본 설계 가정). latency 만 동기 호출(할인 없음)이고 나머지는 Batch 50% 할인. 채점 요청 수는 후보 요약이 모두 정상(ok)이라고 본 상한이다. 가격표 config/pricing.yaml(확인일 2026-10-20). 빌드할 때 tools/budget.py 의 budget_lines 를 실행해 얻은 값이며, 파일럿 뒤 계획기가 count_tokens 와 실제 출력 통계로 다시 계산한다. 상한에서 합계를 뺀 나머지는 예비(실패 재시도 · 채점 불가 재채점 · 추정 오차)로 남기고 단계에 배분하지 않는다. 금액은 소수 둘째 자리 반올림이라 단계 금액의 합과 합계가 마지막 자리에서 1 다를 수 있다(1센트).

3. 사전 등록표 — δ 와 부트스트랩

아래 값은 실행 전에 업무 YAML 과 이 문서에 같이 적어 고정한다(REQ-051). 계획을 만들 때 계획 해시에 포함되며, run 시작 뒤 값이 달라지면 `report` 가 기본값 선정을 거부한다(POL-13).

업무	품질 지표	δ 초기값	부트스트랩	신뢰구간	시드
T1	정확도	0.03	10,000회	95%(상한 사용)	고정
T2	필드 단위 F1(micro)	0.03	10,000회	95%(상한 사용)	고정
T3	루브릭 통과율	0.05	10,000회	95%(상한 사용)	고정

업무 이름: T1 문의 분류 · T2 정보 추출 · T3 요약. 시드의 값 자체는 업무 YAML 에 적는다(D04). δ 의 단위는 품질 지표의 절대 차이(정확도 · F1 · 통과율은 0~1)다. 초기값은 근거를 적은 출발점이며, 바꾸려면 새 계획 · 변경 이력을 거친다.

4. 선정 규칙 가상 예시 — choose_default 실행 결과

> **가상 예시** — 이 표의 30건 결과는 실제 실측이 아니다. 고정 시드(20261020)로 만든 합성 데이터이며, 비용은 2장의 Batch 단가에 모든 후보가 같은 토큰 수(입력 1,000 · 출력 700)를 쓴다고 가정해 계산한다 — 그래서 Haiku 의 effort 높음 · 낮음 비용이 같다(실제로는 effort 마다 출력 토큰이 다르다). 실측 수치는 D09 에서 다룬다. 표의 판정은 빌드할 때 `choose_default` 를 실행해 얻은 값이다.

후보 4개의 공통 30건 결과: claude-opus-5-5/low 28건 · claude-sonnet-5-5/medium 28건 · claude-haiku-5-5/high 27건 · claude-haiku-5-5/low 24건 정답.

δ = 0.03 일 때

조합	정답	정확도	1,000건당 비용	(B – C) 상한	동급	결과
claude-opus-5-5/low	28/30	0.933	$9.0000	— (B)	동급	B · 기본값 선택
claude-sonnet-5-5/medium	28/30	0.933	$4.5000	0.1000	아님	
claude-haiku-5-5/high	27/30	0.900	$0.2250	0.1000	아님	
claude-haiku-5-5/low	24/30	0.800	$0.2250	0.2667	아님	

δ = 0.10 일 때

조합	정답	정확도	1,000건당 비용	(B – C) 상한	동급	결과
claude-opus-5-5/low	28/30	0.933	$9.0000	— (B)	동급	B(최고 품질)
claude-sonnet-5-5/medium	28/30	0.933	$4.5000	0.1000	동급	
claude-haiku-5-5/high	27/30	0.900	$0.2250	0.1000	동급	기본값 선택
claude-haiku-5-5/low	24/30	0.800	$0.2250	0.2667	아님	

해석

- 같은 결과인데 δ 가 0.03 이면 **claude-opus-5-5/low**, 0.10 이면 **claude-haiku-5-5/high** 가 기본값이다. 30건에서는 신뢰구간이 넓어 δ 에 따라 결론이 바뀐다(spec §6-3 규칙 4).
- 30건에서 한 건은 정확도 0.033 이다. 오답 한 건만 다른 후보도 짝지은 부트스트랩의 상한은 0.10 근처까지 올라가므로, δ 0.03 은 사실상 최고 조합과 결과가 같은 후보만 동급으로 받고 δ 0.10 은 한 건 차이까지 동급으로 받는다.
- 그래서 δ 는 결과를 보기 **전에** 정해야 한다(POL-13). 결과를 본 뒤 고를 수 있다면 어느 조합이든 원하는 쪽이 뽑히도록 δ 를 맞출 수 있다. δ 0.03 이 옳다거나 0.10 이 옳다는 뜻이 아니다 — 이 예시는 판정이 δ 에 민감하다는 것과, 그 민감도를 사전에 못 박는 이유를 보여 준다.
- 표본이 30건이므로 이 판정은 "공통 30건 기준 판정 — 확정에는 추가 표본 필요"다(POL-14 규칙 4).

- δ 0.10 표의 **동률**: claude-sonnet-5-5/medium 은 B(claude-opus-5-5/low)만 맞힌 항목 1건 · 자기만 맞힌 항목 1건(순 차이 0건)이고, 짝지은 부트스트랩 상한 0.1000(재표본에서 3건 차이 ÷ 30)가 δ 와 정확히 같다; claude-haiku-5-5/high 은 B(claude-opus-5-5/low)만 맞힌 항목 1건 · 자기만 맞힌 항목 0건(순 차이 1건)이고, 짝지은 부트스트랩 상한 0.1000(재표본에서 3건 차이 ÷ 30)가 δ 와 정확히 같다. 규칙이 "이하"이므로 동급에 포함된다. 부동소수점 잡음 덕이 아니다 — 구현은 상한 ≤ δ + 1e-9 로 판정해 반올림 오차만 흡수한다(POL-14).

5. 예산·장부

POL-01 실측 전체 예산 상한 5달러

구분	내용
규칙	예산 상한은 실행 한 번이 아니라 프로젝트 전체 누적 장부에 적용한다. 파일럿 · 본실행(R1 · R2) · 채점 · 속도 측정의 합계가 $5 를 넘지 않는다. 단계마다 따로 $5 를 쓰지 않고, 계획을 만들 때와 배치를 낼 때마다 누적 사용액을 읽어 남은 예산 = $5 − 누적 사용액으로 판정한다. 단계별 금액은 손으로 정하지 않는다 — 이 문서 2장의 단계별 예산 표가 spec 의 배분을 가격표로 계산해 보이며(목표치), 상한이 아니다. 현재 배분(2026-10-20 사용자 결정 — 채점은 T3 공통 30건만 Sonnet 5.5 · high, R2 는 Sonnet low/medium · Opus low)의 합계가 상한 안에 드는지와 남는 예비(실패 재시도 · 채점 불가 재채점 · 추정 오차용, 단계에 배분하지 않음)도 그 표가 보인다.
이유	평가 비용을 통제하지 못하면 평가를 하지 않게 된다는 것이 이 도구의 출발점이다. 상한이 실행 단위면 여러 번 나눠 돌려 총액이 쉽게 넘는다. 크레딧은 프로젝트 전체에서 하나이므로 장부도 하나여야 한다.
예시	가상 예시 — 누적 사용액이 $4.9 일 때 보수 금액 $0.2 짜리 계획은 남은 예산($0.1)을 넘으므로 만들어지지 않거나(공통 표본을 줄일 수 없을 때) 줄어든 뒤에야 만들어진다. 앞 단계에서 얼마를 썼든 같은 판정이다.
관련 요구사항	REQ-014, REQ-015, REQ-016, REQ-038, REQ-054

POL-02 실제 비용은 응답 usage 로 재계산해 장부에 기록

구분	내용
규칙	장부에 쌓는 금액은 예상치가 아니라 배치 결과의 `usage`(입력 · 출력 토큰)에 가격표 단가를 곱하고 Batch 이면 50% 할인을 적용해 다시 계산한 실제 비용이다. 배치 하나가 끝날 때마다 한 줄(단계 · 배치 ID · 실제 비용 · 누적 합계)을 더한다. 제출 자체가 실패하면 장부에 기록하지 않는다. 프롬프트 캐싱은 쓰지 않으므로 usage 의 cache 토큰은 0 이어야 한다. 0 이 아니면 오류다 — run 은 그 결과의 cache 토큰을 장부 줄에 함께 기록하고(장부에서 조용히 빼지 않는다) 다음 배치를 내지 않은 채 계획을 halted 로 멈추며 무엇이 · 몇 토큰인지 보고한다. pricing.yaml 에 캐시 단가가 없어 그 비용을 확정할 수 없기 때문이다. 출력 토큰에 thinking 토큰이 포함되는 것은 `구현 1단계에서 확인함`(근사 확인). 콘솔 청구 내역과 장부 합계는 단계마다 대조한다.
이유	예상은 파일럿 3건에서 나온 값이라 틀릴 수 있다. 상한 판정과 예산 보고가 모두 실제 사용량에서 나와야 상한이 의미를 가진다. 가격은 코드에 박지 않고 설정 파일에서 읽는다(POL-18).
예시	Haiku 응답의 `usage` 가 입력 1,000 · 출력 700 토큰이면 장부에는 이 토큰 수로 계산한 금액을 Batch 할인 적용해 적는다. 같은 요청을 동기 호출하면 두 배다 — 숫자는 이 문서 2장 표가 `request_cost` 를 실행해 보여 준다.
관련 요구사항	REQ-020, REQ-028, REQ-034, REQ-038

6. 승인 게이트

POL-03 승인 전 과금 호출 0건

구분	내용
규칙	승인이 기록되기 전에는 과금 호출 0건이다. 과금되는 호출은 하지 않는다 — Batch 제출 · 동기 Messages 호출 · 채점 요청 모두 포함한다. `count_tokens` 는 무료이므로 승인 전에 써도 된다. 승인이 없거나 승인 해시가 현재 계획과 다르면 어댑터의 `submit` 은 호출되지 않는다.
이유	"먼저 얼마가 드는지 보고, 그다음에 결정한다"가 승인 게이트의 존재 이유다. 예외를 하나라도 두면 사전 통제라고 부를 수 없다. 라이브 스모크 테스트도 같은 게이트를 거친다.
예시	`plan` 이 끝난 뒤 사용자가 승인 화면을 닫으면 그 시점까지 과금은 0이다. 계획 파일과 `count_tokens` 결과만 남는다.
관련 요구사항	REQ-022, REQ-024, REQ-031, REQ-042, REQ-056, REQ-057

POL-04 승인은 계획 해시에 묶이고 승인된 계획만 실행

구분	내용
규칙	계획은 파일로 고정되고 내용의 해시를 가진다(조합표 · 건수 · 금액 3종 · 가격표 확인일 · 선정 규칙의 δ 와 부트스트랩 설정 포함). 승인은 그 해시에 기록된다. 계획이 바뀌면 해시가 바뀌므로 다시 승인해야 한다. `run` 은 승인된 해시의 계획만 실행하며, 줄인 계획 · 재시도 계획도 새 계획이므로 새로 승인받는다.
이유	승인과 실행 사이에 계획이 몰래 바뀌면 사용자가 본 금액과 실제 과금이 달라진다. 해시에 묶으면 "무엇을 승인했는가"가 하나로 정해진다.
예시	승인한 뒤 가격표를 갱신하거나 표본을 한 건 줄이면 해시가 달라져 `run` 이 거부되고, 새 계획을 만들어 다시 승인하라는 안내가 나온다.
관련 요구사항	REQ-017, REQ-021, REQ-022, REQ-023, REQ-024, REQ-027, REQ-030

7. 비용 추정

POL-05 예상 비용은 count_tokens 로 계산해 예상·보수·최악 3종으로 낸다

구분	내용
규칙	입력 토큰은 `count_tokens` 로 (모델, 프롬프트 버전, 항목) 단위로 세고 캐시한다(effort 는 입력에 영향이 없다). 출력 토큰은 파일럿 결과로 낸 (모델, effort, 업무)별 통계로 가정해 금액을 세 가지로 계산한다 — 예상 = 출력 토큰 평균 · 보수 = 출력 토큰 p90 · 최악 = `max_tokens` 전부. 승인 화면에는 세 금액을 모두 보여 준다. `count_tokens` 는 출력 스키마 · effort 설정을 받고 스키마 토큰(+213)이 입력에 포함된다는 것을 구현 1단계에서 확인함.
이유	출력 토큰은 thinking 때문에 크게 흔들리므로 한 숫자로는 위험을 말할 수 없다. 평균은 기대치, p90 은 통과 기준용, `max_tokens` 는 넘을 수 없는 상한이다.
예시	승인 화면의 세 금액은 항상 예상 ≤ 보수 ≤ 최악 순이다. 어느 한 칸만 크게 벌어져 있으면 파일럿 표본(3건)이 작아 통계가 불안정하다는 신호로 읽는다.
관련 요구사항	REQ-013, REQ-014

POL-06 예상치와 실제 비용을 비교해 보고

구분	내용
규칙	단계가 끝나면 리포트에 계획 당시의 예상 · 보수 금액과 장부의 실제 비용을 나란히 적고 오차를 보고한다. 오차가 크더라도 규칙은 바뀌지 않으며(POL-13) 다음 단계 계획의 입력으로만 쓴다.
이유	계획기가 얼마나 맞는지는 이 도구의 성과 지표다. 틀린 추정을 숨기면 승인 게이트가 주는 안심이 근거를 잃는다.
예시	실제 비용이 보수 금액을 넘었다면 리포트에 그렇게 적고, 파일럿 표본이 작아서였는지(3건) 입력 토큰 추정이 모자랐는지를 한계로 남긴다.
관련 요구사항	REQ-018, REQ-049

POL-21 통과 기준은 보수 ≤ 남은 예산(파일럿은 최악), 넘으면 정해진 순서로 줄인다

구분	내용
규칙	계획의 통과 기준은 보수 금액 ≤ 남은 예산이다. 파일럿은 아직 출력 통계가 없으므로 최악 금액 ≤ 남은 예산을 기준으로 한다. 넘으면 계획기가 stage 마다 정해진 순서로 줄이고, 줄일 때마다 기준을 다시 본다. stage 별 순서 — R1: Haiku 비공통 표본(31~100번째)을 줄인다(R1 에서 줄일 수 있는 것은 이것뿐이다). R2: 줄이지 않는다 — 이긴 버전 하나 · Sonnet low/medium · Opus low × 공통 30건이 이미 최소 배분이다. judge-R1 · judge-R2: 줄이지 않는다 — 채점 대상은 T3 공통 30건으로 고정이다. latency: 조합마다 측정 건수를 3 → 2 → 1 건 순으로 줄인다. 0건으로는 줄이지 않는다. pilot: 줄이지 않는다 — 최악 금액이 남은 예산을 넘으면 계획을 만들지 않는다. 공통 30건은 어느 stage 에서도 줄이지 않는다. 줄일 것이 없거나 다 줄여도 넘으면 계획을 만들지 않고 이유를 출력한다.

구분	내용
이유	보수(p90)는 "대체로 이 안에 들어온다"는 기준이지 보장이 아니다. 파일럿은 통계가 없으니 가장 나쁜 경우를 쓴다. 줄이는 순서는 비교의 근거(모든 조합이 같은 30건을 공유)를 건드리지 않는 것만 줄이도록 정했다 — Haiku 의 추가 70건은 Haiku 내부 비교를 정밀하게 할 뿐이고, latency 건수는 지연 p50 · p90 의 정밀도만 바꾼다(1건이면 거친 값이지만 조합 사이 비교는 남는다). 채점 · R2 를 줄이면 공통 30건 비교 자체가 깨지므로 줄이지 않는다.
예시	가상 예시 — R1 계획에서 남은 예산이 $0.3 인데 보수 금액이 $0.4 면 Haiku 31~100번째 표본부터 줄여 다시 계산하고, 공통 30건만 남겨도 $0.3 을 넘으면 계획을 만들지 않고 "공통 표본 비용이 남은 예산 초과"를 출력한다.
관련 요구사항	REQ-014, REQ-015, REQ-016

8. 배치 분할·정지

POL-07 배치는 순서대로 제출하고 다음 배치가 상한을 넘기면 제출 전 정지

구분	내용
규칙	계획은 작은 배치 여러 개(크기는 POL-20)로 나뉘어 한 번에 하나씩 순서대로 제출된다. 배치가 끝나 장부가 갱신된 뒤에야 다음 배치를 낸다. 다음 배치의 보수 금액을 누적 사용액에 더해 $5 를 넘으면 제출 전에 멈추고 계획은 halted 가 된다. 정지 시점에 과금 호출은 하지 않는다.
이유	한꺼번에 내면 상한 판정을 할 기회가 없다. 순차 제출은 "이미 쓴 돈"을 알고 나서 다음 지출을 결정하게 만든다. 보수 금액은 p90 이므로 실제가 그보다 클 수 있다는 점은 남는다 — 그 차이는 배치 크기만큼으로 제한된다.
예시	가상 예시 — 누적 $4.7 인 상태에서 보수 $0.5 짜리 배치가 남아 있으면 합계 $5.2 > $5 이므로 제출하지 않고 멈춘다. 이미 끝난 배치의 결과와 장부는 그대로다.
관련 요구사항	REQ-027, REQ-029

POL-08 정지 시 남은 요청은 pending 으로 보존

구분	내용
규칙	상한 정지 때 아직 내지 않은 요청은 삭제하지 않고 `pending` 상태로 남긴다. 끝난 배치의 결과는 그대로 집계에 쓸 수 있다. 정지(상한 정지 · 프로세스 중단 모두)된 계획은 같은 계획 해시로 `run --resume` 해서 이어 간다 — 새 승인은 필요 없다. `run` 은 시작할 때 계획에 소유자(잠금 · pid)를 기록한다. `--resume` 은 halted 계획과, running 인데 소유자가 살아 있지 않은 계획(강제 종료로 halted 를 기록하지 못한 것 — 먼저 halted 로 바꾼다)을 받고, 소유자가 살아 있는 running 계획은 거부한다. 재개는 먼저 제출됐지만 결과를 받지 않은 배치를 DB 의 배치 ID 로 조회 · 수집해 실제 비용을 장부에 기록한다(이미 낸 배치라 새 과금 호출이 아니다). 그 뒤에 남은 예산 ≥ 다음 배치의 보수 금액인지 보고, 모자라면 halted 로 남는다. 이미 낸 배치는 다시 내지 않는다.
이유	어디까지 했는지가 남아야 같은 일을 두 번 하고 두 번 과금하는 사고가 없다. 재개는 승인된 계획을 그대로 이어 가는 것이라 승인이 다시 필요하지 않고, 상한은 재개 때도 제출 전 검사로 지켜진다. 받지 않은 배치의 비용을 장부에 넣기 전에 상한을 보면 이미 쓴 돈을 빼먹고 판정하게 된다 — 그래서 수집이 먼저다. 소유자 검사는 강제 종료된 계획을 이어 갈 수 있게 하면서, 살아 있는 실행과 겹쳐 같은 배치를 두 번 내는 일을 막는다.
예시	계획의 요청 중 앞쪽 배치가 끝나고 뒤쪽이 pending 이면, 리포트는 "계획 일부 실행 — 정지"로 표시하고 집계에서 pending 건수를 따로 적는다. 이어 하려면 `run <plan_id> --resume` (같은 해시, 남은 예산 ≥ 다음 배치 보수 금액일 때만).
관련 요구사항	REQ-029, REQ-030

POL-20 배치 하나는 요청 200건 또는 보수 금액 $0.50 중 먼저 닿는 쪽까지

구분	내용
규칙	계획을 배치로 나눌 때 한 배치에 담는 요청은 200건 이하이고 그 배치의 보수 금액 합이 $0.50 이하다 — 둘 중 먼저 닿는 쪽에서 배치를 끊는다. 이 크기는 초기값이며 실측 후 조정한다(배치 처리 시간 · 실제 비용 오차를 본 뒤 계획 B 에서 값을 정하고, 바꾸면 새 계획 해시로 승인받는다). 요청 하나의 보수 금액이 $0.50 을 넘으면 그 요청 하나가 한 배치다.

구분	내용
이유	상한 정지의 해상도 대 제출 횟수의 맞바꿈이다. 배치가 작을수록 상한에 가깝게 멈추고 정지 때 남기는 미사용 예산이 줄지만, 제출 · 조회 횟수가 늘고 배치마다 대기가 생긴다(배치는 최대 24시간). 배치가 크면 반대다. $0.50 은 상한 $5 의 10% 로 정지 때 남는 예산의 최대치를 거기에 묶고, 200건은 가장 싼 조합(Haiku)에서 배치 수가 지나치게 많아지지 않도록 한 값이다. 둘 다 정답이 아닌 출발점이다.
예시	가상 예시 — 보수 금액 합이 $1.20 인 계획이 요청 대부분 $0.002 짜리라면, 먼저 닿는 쪽은 200건($0.40)이므로 3개 이상의 배치로 나뉜다. 요청당 $0.01 짜리 비싼 조합이면 50건($0.50)에서 끊긴다. 이 숫자는 규칙을 설명하는 가상 값이다.
관련 요구사항	REQ-027

9. 실패 처리

POL-09 실패 항목은 오답 처리하고 유형별 건수 보고

구분	내용
규칙	평가 대상(후보) 쪽의 실패 — 배치 항목 errored · expired · canceled, 거절(refusal), `max_tokens` 잘림, 형식 파싱 실패 — 는 모두 오답으로 집계하고 유형별 건수를 따로 보고한다. 채점자 쪽 실패(채점 요청의 오류 · 거절 · 파싱 실패)는 후보의 오답이 아니라 "채점 불가"로 따로 센다(POL-22).
이유	실패를 집계에서 빼면 실패를 잘 숨기는 조합이 높은 점수를 받는다. 실제로 쓸 때 실패는 쓸모 없는 응답이므로 오답이 맞다. 반대로 채점자의 실패까지 후보 탓으로 돌리면 비교가 왜곡된다.
예시	어떤 조합이 100건 중 5건이 `max_tokens` 로 잘렸다면 그 5건은 오답이고, 결과표에 "잘림 5"가 따로 적힌다(가상 예시).
관련 요구사항	REQ-041, REQ-046, REQ-059

POL-10 자동 재시도 없음 — 재시도는 새 계획을 만들어 다시 승인

구분	내용
규칙	자동 재시도는 없다 — 도구가 실패 항목을 스스로 다시 보내지 않는다. `run --retry-failed` 의 대상은 인프라 실패(배치 항목 errored · expired · canceled)와 채점 불가(채점자 쪽 실패, POL-22) 두 가지뿐이며, 그 항목만 담은 새 계획을 만들 뿐이다. 새 해시를 가지므로 다시 승인해야 실행된다. 거절(refused) · `max_tokens` 잘림 · 형식 파싱 실패는 모델이 낸 답이므로 확정된 오답이고 재시도하지 않는다(POL-09). 재시도 결과는 같은 (조합, 항목)의 실패 결과를 대신해 집계에 들어간다. 실패 행은 지우지 않는다. 계보는 두 갈래다 — 인프라 실패로 다시 보낸 후보 요청은 새 결과 행의 `retry_of` (대신한 요청의 request_id)에, 채점 불가를 다시 채점한 것은 같은 결과에 붙는 새 `judgments` 행(`judgment_seq` 증가 · 그 재채점 계획의 `plan_id`)에 남는다(D04). 정지된 계획을 이어 가는 `--resume` 은 재시도가 아니며 새 계획이 아니다(POL-08). 429 · 5xx · 연결 오류는 SDK 의 기본 재시도와 백오프에만 맡기고, 제출 자체가 실패하면 장부에 기록하지 않는다.
이유	자동 재시도는 사용자가 승인하지 않은 과금이다. 새 계획을 거치면 재시도 비용도 누적 장부와 보수 금액 판정을 받는다. 인프라 실패는 모델의 답이 아니므로 다시 받아도 비교가 공정하다. 반대로 거절 · 잘림 · 파싱 실패를 다시 보내 다른 답을 받으면 "맞을 때까지 돌리기"가 되어 실패를 오답으로 세는 규칙(POL-09)과 사전 등록(POL-13)이 무너진다.
예시	`run --retry-failed` 를 실행하면 "새 계획 생성 — 승인 필요"에서 멈추고 과금 호출은 없다. 사용자가 승인하면 그 계획만 실행된다.
관련 요구사항	REQ-031

10. 채점

POL-11 업무별 품질 지표로 채점

구분	내용
규칙	T1 은 라벨 완전 일치 정확도(macro-F1 도 보고), T2 는 필드 단위 micro F1(레코드 완전 일치율도 보고 — 값은 NFC · 앞뒤 공백 · 연속 공백 정규화 뒤 (슬롯 종류, 값) 쌍으로 비교), T3 는 LLM 채점자 루브릭 통과율이다. 기본값 선정(POL-14)에 쓰는 지표는 T1 정확도 · T2 필드 F1 · T3 통과율이다.
이유	업무마다 "맞았다"의 뜻이 다르다. 선정에 쓰는 지표를 업무당 하나로 정해 두어야 규칙을 미리 고정할 수 있다.
예시	T2 에서 (slot, 값) 두 쌍 중 하나만 맞으면 필드 단위로는 부분 점수를 받지만, 레코드 완전 일치율에서는 틀린 것이다 — 둘 다 리포트에 있다.
관련 요구사항	REQ-039, REQ-040

POL-12 교정 일치율이 80% 미만이면 T3 결과를 참고용으로 표시

구분	내용
규칙	`calibrate` 로 사람이 교정 세트 30개를 같은 루브릭으로 채점한다(과금 없음). 채점자와 사람의 통과/실패 일치율이 80% 미만이면 T3 결과에 "참고용" 표시를 붙이고 루브릭 개선을 한계로 남긴다. 80% 이상이면 표시하지 않는다. 일치율과 Cohen's κ 는 어느 경우에나 리포트에 적는다. 이 기준은 표시 여부만 정하며 선정 규칙의 실행을 막지 않는다. 채점자를 실행 전에 "보정"해서 점수를 조정하는 일은 하지 않는다.
이유	사람과 채점자가 자주 어긋나면 T3 점수를 확정된 사실처럼 읽어서는 안 된다. 그러나 30개 교정은 거친 확인이라 결과를 버릴 근거까지는 되지 못한다 — 그래서 막지 않고 표시한다. 자기 선호 편향(채점자 Sonnet 5.5 가 Sonnet 의 요약을 채점)은 별도의 한계로 적는다.
예시	가상 예시 — 교정 30개 중 23개에서 판정이 같으면 일치율 77% 이므로 T3 결과표 위에 "참고용" 배지가 붙고, 어느 항목에서 어긋났는지(이유 비교)가 한계 절에 적힌다.
관련 요구사항	REQ-042, REQ-043, REQ-044, REQ-045

POL-22 채점자는 고정하고 블라인드로 채점하며, 채점자 쪽 실패는 채점 불가로 센다

구분	내용
규칙	T3 채점자는 Sonnet 5.5(`claude-sonnet-5-5`)의 effort high 로 고정한다(2026-10-20 사용자 결정으로 Opus 5.5 에서 바꿨다 — 예산 상한 $5 안에 채점을 넣기 위해서다). 채점 대상은 T3 공통 30건의 요약뿐이다(judge-R1 = R1 의 Haiku 조합, judge-R2 = R2 의 Sonnet · Opus 조합). 루브릭 세 항목(① 사실 일치 ② 핵심 포함 ③ 한국어 1~3문장 형식)을 항목마다 통과/실패와 이유로 판정하고, 세 항목이 모두 통과해야 그 요약이 통과다(전체 통과는 코드가 계산한다). 채점자 입력에는 모델 · effort · 프롬프트 버전을 넣지 않는다(블라인드) — 이것이 채점 편향을 줄이는 장치다. 요약을 채점자에게 내는 순서는 고정 시드로 섞지만, 이는 운영상의 선택일 뿐 편향 대책이 아니다(Batch 의 요청은 서로 독립이라 제출 순서가 판정에 영향을 주지 않는다). 루브릭 항목 순서(①②③)는 고정한다. ③ 형식에서 고유명사의 로마자 표기는 한국어 문장 안에서 허용한다(spec 에는 없는 D08 의 결정이다). 채점 대상은 후보의 `ok` 출력만이다 — 거절 · 잘림 · 실패는 이미 오답이다(POL-09). 채점자 쪽 실패(채점 요청의 오류 · 거절 · 파싱 실패)는 후보의 오답이 아니며 "채점 불가"로 따로 센다. 채점 불가는 새 계획을 승인받아야 다시 채점할 수 있고(POL-10), 통과율 계산에서의 분모 처리 세부는 D04 에서 정한다. 기본값 선정의 짝지은 비교는 두 조합 모두 채점된 항목(교집합)에서 하고, 빠진 건수를 쌍마다 보고하며, 교집합이 20건 미만이면 "판정 보류"다(POL-14).
이유	채점자 조건이 바뀌면 T3 점수끼리 비교할 수 없으므로 고정한다. 어느 모델의 답인지 알면 같은 모델이나 더 강한 모델에 후해질 수 있어 가린다. 채점자의 실패를 평가 대상의 책임으로 돌리면 조합 비교가 왜곡된다. 자기 선호 편향(Sonnet 채점자가 Sonnet 후보에 후할 수 있음)은 블라인드로 단서를 줄일 뿐 없애지는 못하며 한계로 명시한다. 채점자가 가장 강한 모델(Opus)이 아니라는 점도 한계다 — 교정 일치율(POL-12)이 그 영향을 확인하는 장치다.
예시	채점 응답이 JSON 으로 파싱되지 않은 요약은 해당 조합의 "채점 불가 1"로 세어 결과표에 나란히 적히고, 오답 건수에는 들어가지 않는다(가상 예시). 로마자로 쓴 인명이 문장에 있어도 ③ 은 실패가 아니다.
관련 요구사항	REQ-042

11. 기본값 선정

POL-13 선정 규칙과 δ 는 실행 전에 사전 등록하고 결과를 본 뒤 바꾸지 않는다

구분	내용
규칙	선정 규칙(POL-14)과 허용폭 δ(초기값 T1 0.03 · T2 0.03 · T3 0.05), 부트스트랩 설정(10,000회 · 95% 신뢰구간 · 고정 시드)은 실행 전에 업무 YAML 과 이 문서에 사전 등록한다. 계획을 만들 때 그 시점의 δ 와 부트스트랩 설정이 계획 해시에 포함되어 기록된다. 업무 YAML 에 δ 가 비어 있으면 계획 생성이 거부된다. 결과를 본 뒤 기준을 바꾸지 않는다. `report` 는 업무마다 업무 YAML 의 현재 δ 를 그 업무를 담은 모든 계획(R1 · R2 · judge-R1 · judge-R2)에 기록된 δ 와 하나씩 비교하고, 하나라도 다르면 기본값 선정을 거부하며 어느 업무 · 어느 계획의 δ 가 얼마에서 얼마로 바뀌었는지 출력한다. 기준을 정말 바꿔야 하면 새 계획으로 새로 실행하고, 변경 이력 표와 이유를 남긴다.
이유	30건 표본에서는 δ 에 따라 결론이 뒤바뀐다(이 문서 4장 예시). 결과를 본 뒤 δ 를 고를 수 있으면 원하는 조합이 뽑히도록 δ 를 맞출 수 있다. 기준을 먼저 적어 두는 것이 "근거 있는 기본값"이라는 주장의 전제다.
예시	T3 결과가 마음에 들지 않아 δ 를 0.05 에서 0.10 으로 고쳐 `report` 를 돌리면 "T3 δ: 계획 0.05 → 현재 0.10, 선정 거부"가 출력된다.
관련 요구사항	REQ-007, REQ-050, REQ-051, REQ-052

POL-14 동급 후보 중 1000건당 비용이 가장 낮은 조합 선택

구분	내용
규칙	업무마다 다음을 순서대로 적용한다. (1) 공통 30건에서 품질 지표가 가장 높은 조합을 B 로 둔다(동점이면 조합 ID 순). (2) 각 후보 C 에 대해 짝지은 부트스트랩(10,000회 · 고정 시드)으로 (B − C) 의 95% 신뢰구간을 구해 상한이 δ 이하이면 C 를 동급으로 본다 — 부등호는 "이하"이며, 상한 ≤ δ + 1e-9 로 판정한다(1e-9 는 부동소수점 반올림만 흡수하는 값이고 판정 여유가 아니다). (3) 동급 후보(B 포함) 중 1,000건당 비용이 가장 낮은 조합이 기본값이다. 같으면 지연 p50 이 낮은 쪽, 그것도 같으면 effort 가 낮은 쪽이다. 1,000건당 비용 = 그 조합이 실행한 main 항목 전체(Haiku 100건 · Sonnet · Opus 30건)의 후보 요청 비용(실제 `usage` × Batch 단가)의 평균 × 1,000 이다. 채점(judge-R1 · judge-R2) 비용과 latency 단계 비용은 넣지 않는다 — 업무에 그 조합을 쓸 때 드는 비용만 비교한다. (4) 공통 30건 기준 판정이므로 리포트에 "공통 30건 기준 판정 — 확정에는 추가 표본 필요"를 붙인다. (5) T3 는 채점 불가(POL-22) 항목이 조합마다 다를 수 있으므로, B 와 C 의 짝지은 비교는 두 조합 모두 채점 불가가 아닌 항목(교집합)에서 한다. 쌍마다 빠진 항목 수를 결과표에 적고, 교집합이 20건 미만이면 그 C 는 "판정 보류"로 표시하며 동급으로 보지 않는다.
이유	"가장 점수가 높은 조합"만 고르면 늘 가장 비싼 쪽이 뽑힌다. 통계적으로 구분되지 않는 후보 중에서 싼 것을 고르는 것이 이 도구가 보여 주려는 "이 업무엔 이 조합이면 충분하다"의 정의다. 짝짓기(같은 항목끼리 비교)는 항목 난이도 차이를 지워 표본이 작아도 쓸 만하게 한다. 짝짓기는 같은 항목이 양쪽에 있어야 성립하므로 T3 는 교집합에서 비교하고, 남은 항목이 너무 적으면(20건 미만 — 이 문서가 정한 값) 판정하지 않는다. 채점 비용은 조합을 고른 뒤에는 들지 않는 평가 비용이고, latency 는 동기 호출이라 단가가 Batch 와 다르므로 1,000건당 비용에서 뺀다.
예시	이 문서 4장 — 같은 가상 결과에 δ 0.03 과 0.10 을 적용한 표. 실제 `choose_default` 실행 결과다.
관련 요구사항	REQ-047, REQ-050

12. 데이터 라이선스

POL-15 데이터 원본은 커밋하지 않고 라이선스와 출처를 기록

구분	내용
규칙	데이터셋 원본은 저장소에 넣지 않는다. 내려받기 스크립트와 고정 시드로 뽑은 표본 ID 목록만 커밋한다(`runs/` 도 커밋하지 않는다). 데이터셋마다 라이선스와 출처 URL 을 기록해 `prepare` 결과와 리포트에 보인다. T1 · T2(MASSIVE ko-KR)는 CC BY 4.0, T3(XL-Sum Korean)는 CC BY-NC-SA 4.0 — 비상업이다. XL-Sum 은 포트폴리오 · 연구용으로만 쓰고 원문을 재배포하지 않는다.
이유	라이선스 조건을 지키는 가장 단순한 방법은 원본을 배포물에 넣지 않는 것이다. 비상업 데이터가 들어간 결과물을 상업 목적에 쓰면 조건 위반이 된다.
예시	저장소에서 `git ls-files` 로 보면 표본 ID 목록은 있고 XL-Sum 기사 원문 파일은 없다. 리포트의 T3 절에는 "CC BY-NC-SA 4.0 · 포트폴리오/연구용"이 적힌다.
관련 요구사항	REQ-001, REQ-002, REQ-006

13. 인증·비밀

POL-16 ANTHROPIC_API_KEY 만 사용

구분	내용
규칙	인증은 환경 변수 `ANTHROPIC_API_KEY` 하나만 쓴다. 설정 파일 · 명령줄 인자로 키를 받지 않는다(spec 에 없는 D03 의 결정 — 키가 설정 파일이나 셸 기록에 남는 경로를 없애기 위해서다). 키가 없으면 과금 호출 전에 알리고 멈춘다.
이유	키를 받는 경로가 하나면 새어 나갈 곳도 하나다. 실측은 Console API 크레딧으로 하며 Claude 구독과는 별개다.
예시	키 없이 `promptbench run` 을 실행하면 "ANTHROPIC_API_KEY 가 없다"는 안내가 나오고, 승인 여부와 관계없이 아무것도 제출되지 않는다.
관련 요구사항	REQ-055

POL-17 키는 로그·DB·리포트에 남기지 않음

구분	내용
규칙	API 키 값은 로그 · SQLite DB · 원본 JSONL · 리포트 · 오류 메시지 어디에도 기록하지 않는다. 키를 요청 헤더에 붙이는 일은 SDK 에 맡기고 도구 코드는 값을 읽어 문자열로 보관하지 않는다.
이유	`runs/` 와 리포트는 공유될 수 있는 산출물이다. 한 번 파일에 들어간 키는 되돌릴 수 없다.
예시	API 오류로 예외가 났을 때 장부와 로그에는 오류 유형과 상태 코드만 남고 요청 헤더 · 키 값은 남지 않는다.
관련 요구사항	REQ-037, REQ-055

14. 가격표 관리

POL-18 가격표는 설정 파일로 두고 확인일 기록

구분	내용
규칙	모델 단가 · Batch 할인율 · 확인일(`verified_on`) · 출처 URL 은 `config/pricing.yaml` 한 곳에만 두고 코드에 가격을 적지 않는다. 가격이 바뀌면 이 파일만 고치고 확인일을 갱신한다. 가격표가 바뀌면 계획 해시가 달라지므로 다시 승인한다 (POL-04). Haiku 5.5 의 입력 10만 토큰 초과 구간은 계획마다 `long_context_requests` 경고로 검사하며, 걸리는 요청이 있으면 초과 구간 단가로 계산하고 승인 화면에 표시한다.
이유	가격은 바뀌고, 이 도구의 결과는 가격 위에서만 의미가 있다. 어느 날짜의 가격으로 계산했는지가 남아야 나중에 결과를 해석할 수 있다.
예시	현재 `pricing.yaml` 의 확인일은 2026-10-20, 출처는 공식 가격 페이지다. 1건당 비용 예시(2장)는 이 파일을 읽어 계산한다.
관련 요구사항	REQ-019

POL-19 확인일이 30일을 넘으면 승인 화면에 경고

구분	내용
규칙	가격표 확인일로부터 30일을 넘기면(31일째부터) 승인 화면에 경고를 표시한다. 경고는 차단하지 않는다 — 사용자는 확인하고 승인할 수 있다.
이유	오래된 가격으로 낸 예상 금액은 틀릴 수 있지만, 그것만으로 실행을 막으면 가격이 바뀌지 않은 경우에도 일을 못 한다. 알려주고 사람이 판단하게 한다.
예시	확인일 2026-10-20 의 가격표는 2026-11-19 까지 경고가 없고 2026-11-20 의 승인 화면부터 경고가 뜬다.
관련 요구사항	REQ-025